

tsw-steam-bridge

User Manual

September 19, 2026

A Zig bridge between Steam Input and Train Sim World: it reads real Steam Input digital/analog actions and drives Train Sim World's cab controls directly through its external HTTP API, instead of going through Steam Input's keyboard/mouse emulation.

Why this exists: keyboard emulation flattens two things Train Sim World needs kept apart. Throttle and brake want continuous values, not just “tapped” or “held”. And safety controls like AWS (Automatic Warning System) reset and DSD (Driver's Safety Device / vigilance) reset are separate physical controls in the real cab that often end up sharing or fighting over the same key in a simple keybind scheme. This bridge keeps them independent by talking to Steam Input's real action API and to Train Sim World's real control API, with nothing synthetic in between.

Contents

1	Overview	3
2	Prerequisites	3
3	Enabling Train Sim World's external API	3
4	Vendoring the Steamworks SDK	4
5	Building	4
6	tsw-cli — exploring and testing the API	5
6.1	Checking the API is reachable	5
6.2	Listing controls	5
6.3	Reading and writing values	5
6.4	Building a locomotive profile	5
6.5	Worked example: two real locomotive profiles	6
7	steam-bridge — running the live bridge	6
8	Binding physical controls in Steam	7
9	tsw-gui — the live dashboard	8
10	Troubleshooting	10
11	Reference: profile file format	10

1 Overview

The project has three executables, built from the same source tree:

Executable	Purpose
<code>tsw-cli</code>	Pure Zig, no external dependencies. Talks to Train Sim World’s <code>-HTTPAPI</code> over plain HTTP. Use it to check the API is reachable, explore a locomotive’s control tree, and hand-test individual values.
<code>steam-bridge</code>	The actual bridge process. Links against the Steamworks SDK to read real Steam Input actions and writes their values into Train Sim World via the same client code <code>tsw-cli</code> uses.
<code>tsw-gui</code>	A live dashboard shell built on Dear ImGui, themed from your Omarchy colors and font, with dockable panels.

Control names are **not consistent across locomotives** — not even across versions of the same class — so there is no single, universal “AWS reset” path in Train Sim World’s API. Instead, each locomotive gets its own small JSON *profile* under `profiles/`, mapping a stable set of logical control names (`throttle`, `aws_reset`, `dsd_reset`, ...) onto that particular locomotive’s real node paths. You build these up over time, one locomotive at a time, using `tsw-cli discover`.

2 Prerequisites

- **Zig 0.16.0 or newer.**
- **Train Sim World**, launched with the `-HTTPAPI` Steam launch option (see Section 3).
- **The Steamworks SDK**, vendored locally (see Section 4) — needed to build `steam-bridge`.
- **A Steam client**, running and logged in, for `steam-bridge` and any Steam Input functionality.

`tsw-cli` needs none of the above except Zig itself, so it is the easiest place to start.

3 Enabling Train Sim World’s external API

Train Sim World ships a real, officially-documented external control API — Dovetail’s own *External Interface API v1.5* document (written for TSW6, and confirmed working exactly as described against a live TSW7 session while building this project). To enable it:

1. In Steam, open Train Sim World’s properties and add `-HTTPAPI` to the launch options.
2. Launch the game once. This writes a file called `CommAPIKey.txt` under `Documents/MyGames/TrainSimWorld<N>/Saved/Config/` (where `<N>` is the game’s version number).
3. If you’re running Train Sim World through Proton, that path is inside the Proton prefix, typically under something like
`~/.local/share/Steam/steamapps/compatdata/<appid>/pfx/
drive_c/users/steamuser/MyDocuments/MyGames/....`
The quickest way to find it is:

```
find ~/.steam ~/.local/share/Steam -iname CommAPIKey.txt
```

Every request to the API must include the contents of that file in a `DTGCommKey` HTTP header — the tools in this project handle that for you once you point them at the file with `--key-file`.

The API listens on `http://localhost:31270` — confirmed the same port on a live TSW7 session, build 834; run `tsw-cli info` to confirm it on your own install too (Section 6).

4 Vendors the Steamworks SDK

`steam-bridge` needs Valve’s Steamworks SDK, which is not redistributable and must be downloaded yourself:

1. Sign in with any Steam account at <https://partner.steamgames.com/downloads/list> and download `steamworks_sdk.zip`.
2. Extract it into the project so that `vendor/sdk/public/...` and `vendor/sdk/redistributable_bin/...` exist.

`vendor/sdk/` is git-ignored — it is a large, license-encumbered redistributable, not part of the project’s source. `zig build` will still build `tsw-cli` and `tsw-gui` without it; it only skips `steam-bridge`.

Why app id 480

Steam Input’s API needs to run as some recognized Steam application — `SteamAPI_Init` needs a `steam_appid.txt` file (or a `SteamAppId` environment variable) naming one. For a personal, unpublished tool like this, the standard approach used across the community is Valve’s public “Spacewar” test application, id 480. `steam-bridge` writes `steam_appid.txt` containing 480 automatically on first run if it is missing. If you ever publish this project under your own Steam app id, replace that file’s contents and regenerate `manifest/game_actions_480.vdf` for your id.

5 Building

From the project root:

```
zig build
```

This builds `tsw-cli` and `tsw-gui` unconditionally, and `steam-bridge` if the Steamworks SDK has been vendored (Section 4). The GUI’s own dependencies (`zgui`, `zglfw`, `zopengl`) are fetched automatically the first time you build.

Binaries land in `zig-out/bin/`. You can also run them directly through the build system, which forwards arguments after `--`:

```
zig build run-cli -- info --key-file /path/to/CommAPIKey.txt
zig build run-bridge -- --key-file /path/to/CommAPIKey.txt
zig build run-gui
```

A note if you’re building this yourself on a similar system

On some current Linux systems, Zig 0.16.0’s own linker cannot yet handle the `.sframe` unwind sections that current `glibc/gcc` toolchains emit — any `libc`-linked Zig binary fails to link with an error mentioning `unhandled relocation type R_X86_64_PC64 ... .sframe`. If you hit this, it is a toolchain issue, not a problem with your setup specifically; this project’s own `build.zig` already works around it for `steam-bridge` and `tsw-gui` by linking those two through the system `cc/c++` instead of Zig’s built-in linker.

6 tsw-cli — exploring and testing the API

`tsw-cli` is the tool you reach for first: it talks to Train Sim World directly over HTTP and does nothing with Steam Input at all.

6.1 Checking the API is reachable

```
tsw-cli --key-file /path/to/CommAPIKey.txt info
```

A successful response confirms the port and prints the game's name and build number, which is the fastest way to sanity-check that `-HTTPAPI` took effect and that the port hasn't moved on your version of the game.

6.2 Listing controls

```
tsw-cli --key-file <key> list CurrentDrivableActor
```

Lists the child nodes and endpoints of whatever locomotive you are currently driving — this is how you discover what is actually controllable on a given loco, since names vary by locomotive class.

6.3 Reading and writing values

```
tsw-cli --key-file <key> get CurrentDrivableActor/Throttle_F.InputValue  
tsw-cli --key-file <key> set CurrentDrivableActor/PanUp.InputValue 1
```

The writable endpoint on every interactive control — lever, button, or switch — is always named `InputValue`, confirmed against DTG's own documentation. **It only ever accepts a number, even for a control that's conceptually a boolean:** sending `true/false` fails with `{"Result": "Error", "Message": "Invalid Value (Not a Float)"}` — use `1/0` instead. Use `set` carefully and only while stationary until you know what a control actually does.

Every lever-style control also exposes real, documented functions for its own metadata — worth querying instead of guessing:

```
tsw-cli --key-file <key> get CurrentDrivableActor/Reverser.Function.GetNotchCount  
tsw-cli --key-file <key> get CurrentDrivableActor/Reverser.Function.  
    GetMinimumInputValue  
tsw-cli --key-file <key> get CurrentDrivableActor/Reverser.Function.  
    GetMaximumInputValue
```

Note that `Function.` is part of the endpoint name itself, not a separator you add. **Don't assume every lever's range is 0..1** — confirmed live across two different locomotives that this varies: see the worked example in Section 6.5.

6.4 Building a locomotive profile

```
tsw-cli --key-file <key> discover
```

Walks the current locomotive's writable controls and writes a skeleton profile to `profiles/<ObjectClass>.json`, where `<ObjectClass>` is that locomotive's internal class name. The skeleton lists every candidate writable control under its raw node name; you then:

1. Test each candidate path with `tsw-cli set <path> 1` (or 0, or a float) while parked, and observe what happens in the cab.

2. Rename the JSON key to the matching logical name the bridge expects. The full, current list — covering core cab controls plus British (AWS/DSD/TPWS/DRA), German (SIFA/PZB/LZB) and ETCS actions — is kept in `profiles/README.md` rather than duplicated here, since it grows as the bridge's control vocabulary does.
3. Fix "kind" to "float" for anything that isn't a plain on/off control, and add a "range" override (Section 6.5) if this control's `GetMinimumInputValue`/`GetMaximumInputValue` isn't 0..1.
4. Delete entries you don't care about mapping — an incomplete profile is fine; the bridge simply skips logical controls that aren't mapped for the current locomotive.

6.5 Worked example: two real locomotive profiles

Two locomotive profiles ship with the project already, both built entirely from live TSW7 sessions using the workflow above, not guessed:

- `profiles/RVM_AWL_NT_Class333_DMS0_B_C.json` — British Rail Class 333 (DMS0), a single-cab EMU with one combined power/brake handle. Its `CombinedPowerBrakeHandle.InputValue` runs 0..1 (9 notches, confirmed via `Function.GetNotchCount/GetMaximumInputValue`), and its `Reverser` is a 4-notch lever, also 0..1. No PZB/SIFA/LZB or ETCS mapped — it's a British unit that doesn't carry them.
- `profiles/RVM_AWL_NT_Class331_DMS_C.json` — British Rail Class 331 (DMS), also a combined-handle EMU, but its `PowerBrakeController.InputValue` genuinely reports -1..1 rather than 0..1 (5 notches) — confirmed live by querying that control's own `Function.GetMinimumInputValue/GetMaximumInputValue`, then moving it and watching the value snap to notch positions. This locomotive also needs its `MasterKey` switched on and its `Reverser` moved out of neutral before `PowerBrakeController` has any effect at all — worth checking first on any unfamiliar locomotive if power/brake seems unresponsive.

Discovering the second locomotive's real range is exactly why the profile format has a "range" override field (Section 11) — assuming every lever shares the first locomotive's 0..1 range turned out to be wrong the moment a second one was tested. Always check `GetMinimumInputValue/GetMaximumInputValue` on a new locomotive rather than assuming.

7 steam-bridge — running the live bridge

Once you have a locomotive profile and the Steamworks SDK vendored:

```
zig build run-bridge
```

You don't need `--key-file` by hand: if it's omitted, `steam-bridge` (and `tsw-gui`) search for `CommAPIKey.txt` themselves across every Steam library and Proton prefix they can find (Section 3), picking the most recently modified match if more than one Train Sim World version is installed. Pass `--key-file` explicitly if you need to override that.

On startup, `steam-bridge` initializes the Steamworks API, activates its own `Driving` Steam Input action set (defined in `manifest/game_actions_480.vdf`, entirely separate from Train Sim World's own Steam Input configuration), and begins polling. Every tick it:

1. Reads the current locomotive's class from Train Sim World and loads the matching profile from `profiles/`.

2. Polls each logical Steam Input action — `throttle`, `brake` and `combined_power_brake` as analog axes or stepped detents depending on the active profile's setting for that lever, the rest as digital buttons.
3. For anything that changed and has a mapping in the active profile, sends the new value to Train Sim World via `PATCH /set/...`

Notched vs. notchless levers

Real locomotives have both kinds of throttle/brake handle, and holding an auto-centering analog stick at one precise deflection to maintain line speed for an hour is uncomfortable at best. So each of `throttle`, `brake` and `combined_power_brake` can instead be driven by a pair of up/down detent actions (`throttle_up/throttle_down`, etc.) that `steam-bridge` accumulates into an absolute position itself — snapped to a fixed number of notches, or stepped by a tunable amount for a notchless lever. This is controlled per-lever by that control's "mode" field in its locomotive profile ("absolute", "notched" with a "notches" count, or "notchless" with a "step_size"); see `profiles/README.md` for the full field reference. The default step size for notchless levers can be overridden globally with `steam-bridge --step-size <value>`.

A detent-based binding pairs naturally with Steam Input's trackpad "Scroll Wheel" mode set to circular swipe, which turns a flat trackpad into a haptic rotary encoder — but any pair of buttons or paddles works just as well.

Current status: Steam's own API confirms successful initialization against a real, running Steam client, but activating the custom action manifest is not yet reliably working end-to-end — this is the active area of development. See the project README's "Known gaps" section for the latest status. Note that `tsw-gui` runs this exact same bridge internally (Section 9); running both at once against the same controller and profile isn't useful and will have them race each other's `/set` calls, so pick one or the other.

8 Binding physical controls in Steam

Steam Input has no idea what a `combined_power_brake` action should map to on your specific controller until you tell it, and that binding lives entirely inside Steam's own UI — this project never reads or writes controller bindings itself. It only declares what actions exist (`manifest/game_actions_480.vdf`) and polls whatever Steam reports for them once you've bound them.

Because this is a personal, unpublished tool rather than something published under its own Steam app id, it identifies itself to Steam as Valve's public "Spacewar" test application, id 480 (Section 4). That has one very concrete consequence for you: **the game you configure controller bindings for inside Steam is called "Spacewar," not "Train Sim World."**

1. **Install Spacewar once**, if you haven't already — it's free. From the Steam client, open the Store and search for "Spacewar," or navigate directly to steam://install/480 and install it. This doesn't launch a real game; it's a lightweight test application Valve ships specifically so indie developers can exercise Steam Input without owning a published app id.
2. **Launch steam-bridge or tsw-gui** (either one). On startup it calls `SteamAPI_Init` claiming app id 480, so Steam now treats this project's own process as if it were Spacewar currently running.
3. **Open Steam's controller configurator for Spacewar.** The most reliable path is Big Picture Mode: Steam → View → Big Picture Mode → Library → find Spacewar (it will show as "Running") → the controller/gear icon → Controller Options → Configure. Desktop Steam sometimes offers the same screen via the controller icon in the top-right of the client

while `steam-bridge/tsw-gui` is running, but Big Picture Mode is the version that reliably surfaces a game’s custom action manifest.

4. **Bind every action.** Steam shows the `Driving` action set from `manifest/game_actions_480.vdf` and every action it declares — `combined_power_brake`, `throttle`, `brake`, `reverser`, all four `_up/_down` notch-stepping pairs, and every digital control (`horn`, `aws_reset`, `master_key_toggle`, ...). Bind analog sticks/triggers to the three levers’ absolute axes, any button/paddle to the digital controls, and — for a notched lever’s tactile “rotary encoder” feel — a trackpad set to “Scroll Wheel” (circular) mode bound to that lever’s `_up/_down` pair (Section 7).
5. **Save the configuration.** Steam applies it immediately; `steam-bridge/tsw-gui` reflects your input the moment you move or press a bound control, visible live in `tsw-gui`’s panels (Section 9).

Current status: as noted at the end of Section 7, this project’s own call to activate its custom action manifest is not yet confirmed reliably working end-to-end, independently of everything above — Steam Input’s own initialization and the app-id-480 identification both work, but the custom `Driving` action set may not appear or activate correctly for you yet. If bindings made this way don’t seem to reach `steam-bridge/tsw-gui`, that is the known gap tracked in the project README, not a step you’ve missed.

9 tsw-gui — the live dashboard

`tsw-gui` is a windowed dashboard with dockable panels, built on Dear ImGui, that runs the exact same live bridge as `steam-bridge` internally — it isn’t a separate visualization of a running `steam-bridge`, it drives Steam Input and Train Sim World directly itself, once per rendered frame. Just run it:

```
./zig-out/bin/tsw-gui
```

No arguments are required for a first look: it finds your key file the same way `steam-bridge` does (Section 3) and falls back to a clearly-labeled preview mode if nothing is found or Steam isn’t running, rather than failing to start. Its font is bundled into the binary (JetBrains Mono, SIL OFL 1.1), so it looks right with no setup on any platform.

The panels are generated from the same control list `steam-bridge` uses (`src/controls.zig`), so the dashboard always reflects the actual current set of controls, not a fixed example:

- **Status** — Steam Input/controller connection, active locomotive, and whether a profile was found for it.
- **Power & Brake** — a live progress bar and mode (absolute / notched / notchless) for each of the three levers.
- One tabbed panel per remaining category — Core Cab, British Safety, German Safety, ETCS — listing every control in that category, colored dim if unmapped for the current locomotive, normal if mapped, and in the theme’s accent color while actively pressed.

Figure 1 shows this layout: the Status and Power & Brake panels docked on the left, and the ETCS tab selected among the category panels on the right, themed from the Coaster theme’s palette and set in the bundled JetBrains Mono font.

Panels can be dragged by their title bar to rearrange or dock them differently. The first time you run `tsw-gui` (before an `imgui.ini` exists in the working directory), it builds the layout shown above automatically; from then on, Dear ImGui saves whatever arrangement you leave it in to `imgui.ini` and restores it on every subsequent launch.



Figure 1: tsw-gui's default docked layout: Status and Power & Brake on the left, tabbed category panels on the right.

On Linux, colors are additionally read live from your active Omarchy theme's `colors.toml` at startup rather than hardcoded, so the panel follows whichever theme is active system-wide; elsewhere (or on Linux without Omarchy) it falls back to the same colors shown here. Pass `--theme-file/--font-file` to override either explicitly, or on Linux use `scripts/run-gui.sh` to match your current Omarchy font specifically:

```
./scripts/run-gui.sh
```

Current status: see the note at the end of Section 7 — Steam Input manifest activation isn't reliably working end-to-end yet, so the live values shown will generally read as inactive/zero until that's resolved. Run only one of `tsw-gui` or `steam-bridge` at a time against the same controller and profile.

10 Troubleshooting

Symptom	Likely cause / fix
<code>tsw-cli info</code> fails to connect	Check <code>-HTTPAPI</code> is actually in Train Sim World's launch options and the game has been launched at least once since adding it. Confirm the key file path. Try the base URL override if your game version uses a different port.
Steam Input fails with <code>NoSteamClient</code>	Steam isn't running, or isn't fully started yet. Make sure Steam is open and you are logged in before starting <code>steam-bridge</code> .
<code>zig build</code> skips <code>steam-bridge</code>	The Steamworks SDK hasn't been vendored under <code>vendor/sdk/</code> — see Section 4.
A control does nothing when set	Not every node listed by <code>discover</code> is meaningful — some are read-only telemetry mirrors or apply only to specific train sub-systems. Cross-check against what changes in the cab.
<code>tsw-gui</code> shows the built-in font instead of my font	Launch it via <code>scripts/run-gui.sh</code> , or pass <code>--font-file</code> yourself with a real <code>.ttf/.otf</code> path.

11 Reference: profile file format

Profiles live at `profiles/<ObjectClass>.json`:

```
{
  "objectClass": "RVM_AWL_NT_Class331_DMS_C",
  "displayName": "British Rail Class 331 (DMS)",
  "controls": {
    "combined_power_brake": {
      "path": "CurrentDrivableActor/PowerBrakeController.InputValue",
      "kind": "float",
      "mode": "notched",
      "notches": 5,
      "range": { "min": -1, "max": 1 }
    },
    "reverser": {
      "path": "CurrentDrivableActor/Reverser.InputValue",
      "kind": "float",
      "mode": "notched",
      "notches": 3
    },
    "aws_reset": { "path": "CurrentDrivableActor/AWS_Reset.InputValue", "kind": "bool" }
  }
}
```

(a real, verified entry from `profiles/RVM_AWL_NT_Class331_DMS_C.json` — see the worked example in Section 6.5.)

- `path` — the full API path, always ending in `.InputValue`.

- **kind** — "bool" or "float"; controls how the bridge formats the value it sends to `PATCH/set/...`. TSW itself only ever accepts a number on the wire (1/0 for a bool), regardless of this setting — **kind** only affects how the bridge reads Steam Input's own digital/analog action data for this control.
- **mode**, **notches**, **step_size** — only apply to the three lever controls (**throttle**, **brake**, **combined_power_brake**, or a locomotive's own name for the equivalent, like **reverser**); see Section 7 and `profiles/README.md` for the full mode reference.
- **range** — optional `{"min": ..., "max": ...}` override for a lever's `InputValue` bounds. Defaults to 0..1 (every lever's compiled-in default in `src/controls.zig`) when omitted. **Always check, never assume:** confirmed live that this genuinely varies by locomotive — see Section 6.5.

A logical control with no entry is simply never sent for that locomotive.